Getting started solutions

02-E01 Full insertion trace and separate counts

At j=2, save 1b. Comparison 4>1 is true, giving [□,4a,3c,1d]+1b. At i=0 no key is read. Fill to obtain [1b,4a,3c,1d]. At j=3, save 3c. Comparison 4>3 gives [1b,□,4a,1d]+3c; 1>3 is false. Fill to obtain [1b,3c,4a,1d]. At j=4, save 1d. Comparison 4>1 gives [1b,3c,□,4a]+1d; 3>1 gives [1b,□,3c,4a]+1d; 1>1 is false. Fill to obtain [1b,1d,3c,4a].

Shift counts are (1,1,2), totaling I=4. Key comparisons total 1+2+3=6; guards total 2+2+3=7; array writes total (1+1)+(1+1)+(2+1)=7. Record 1b precedes 1d. One insertion reaches the far left, so $C = I + n - 1 - b = 4 + 3 - 1 = 6$.

02-E02 Inner preservation and a counterexample

For [2a,1b], saving 1b and executing A[2]=A[1] leaves physical storage [2a,2a], which is not the original permutation. Ignore logical hole $h = i + 1$. The correct invariant is $A[1..i] = B[1..i]$, $A[i + 2..j] = B[i + 1..j - 1]$, with right keys greater than key; nonhole records plus key equal B. Initially the right interval is empty. Each shift copies the last left record into the hole and moves the hole left without losing a logical record. At exit, left keys are at most key and right keys greater, so filling gives a sorted permutation.

Using $\geq$ still terminates, orders keys, and preserves records, but moves equal keys right. Input [2a,2b] becomes [2b,2a], losing stability. Its right-interval invariant must use $\geq$ rather than strict >.

02-E03 Shifts count inversions

For fixed right endpoint j, exactly the prefix records greater than S[j] shift once. They are precisely inversions ending at j. Partitioning all inversion pairs by their unique right endpoint proves $\sum h_j = I$. The example gives $h_2 = 1, h_3 = 0, h_4 = 2$, with pairs (a,b), (a,d),(c,d), hence I=3. Under uniform distinct-key permutations, each pair is inverted with probability 1/2, so $E[I] = \binom{n}{2}/2 = n(n - 1)/4$. Linearity also holds for dependent indicators.

With equal keys, strict > never holds. Every identity permutation has I=0, so E[I]=0. Uniform permutation alone does not justify the distinct-key formula.

02-E04 A distribution for average search cost

Position k needs k comparisons. Conditional expected cost on success is $\sum_{k=1}^{n} k/n = (n + 1)/2$. Failure costs n. Therefore

$$E[C] = s(n + 1)/2 + (1 - s)n.$$

Getting started solutions (continued)

For n=8,s=3/4 this is $27/8 + 2 = 43/8 = 5.375$. Worst-case cost remains n, including s=1 when the final position remains possible. For fixed $s \in [0, 1]$, expectation is Θ(n), but the coefficient depends on the distribution. Nonuniform successful positions replace the first term with $s \sum kp_k$.

02-E05 Complete stable MERGE

The comparisons at (i,j) are (1,1), selecting 0a; (2,1), selecting 1d; (2,2), selecting 3b on equality; (3,2), selecting 3e; and (3,3), selecting 6f. Output prefixes are [0a], [0a,1d], [0a,1d,3b], [0a,1d,3b,3e], and [0a,1d,3b,3e,6f]. R is exhausted; append 7c. There are 5 comparisons, 6 buffer copies, and 6 output writes.

Right priority changes output to [0a,1d,3e,3b,6f,7c]. It remains ordered and a permutation, satisfying sorting without stability, but fails stable sorting. Five comparisons attain the m−1 bound.

02-E06 Sentinels and aliases have assumptions

The buffers become [M_real,M_sentinel] and [M_real,M_real,M_sentinel]. With left priority, output 1 selects L[1], output 2 selects the left sentinel, and output 3 attempts L[3], outside L. A sentinel equal to a legal key may be selected prematurely.

For views of [2,4,1,3], choosing right-head 1 first writes A[1]=1, changing the unread left-head view from 2 to 1. The next selection reads this overwritten 1 and duplicates data. Use independent copies and length tests; after one side empties, read only the other. A sentinel alternative needs a tag ranking above all real records, not merely the greatest legal key.

02-E07 Odd lengths and peak space

The root is (1,3,5). Its left call is (1,2,3), whose left call (1,1,2) merges to [2b,6a]. The three-record merge gives [2b,5c,6a]. The right call (4,4,5) gives [1d,4e]. The root gives [1d,2b,4e,5c,6a]. Merge sizes 2,3,2,5 total 12 buffer copies and 12 writes.

The largest individual merge buffers 5 records. Child buffers are released before parent merging, giving peak buffer demand 5, distinct from cumulative 12. The deepest live call chain has lengths 5→3→2→1, four calls. Generally stack space is Θ(log n) and record-buffer space Θ(n).

Getting started solutions (continued)

02-E08 Exact recurrence solutions

Let h=lg n. Every internal depth of U costs 5n, and leaves cost 3n, giving $U(n) = 5n \lg n + 3n$, hence U(8)=120+24=144. At depth ℓ, W costs $2^\ell(n/2^\ell - 1) = n - 2^\ell$. Summing depths 0 through h−1 gives $W(n) = n \lg n - n + 1$, hence W(8)=17. Substitution and the n=1 base cases also establish induction.

Actual MERGE comparisons depend on exhaustion positions. W uses m−1 for each merge, a worst-case recurrence. Sorted eight-record input uses 4+4+4=12 comparisons rather than 17. Buffer copies and writes still total 8 per internal level; fewer comparisons do not remove all other work.

02-E09 Θ bounds beyond powers of two

Balanced floor/ceiling splits give depth-ℓ interval sizes between $\lfloor n/2^\ell \rfloor$ and $\lceil n/2^\ell \rceil$, until singleton termination. Height is at most ceil(lg n). At each depth the unfinished intervals are disjoint and have total size at most n. Internal cost at most a fixed c_U times size gives O(n log n).

For $\ell < \lfloor \lg n \rfloor$, every interval has size at least 2, so all are internal and cover n records. Each such level costs at least $c_L n$. There are floor(lg n) such levels, at least lg n/2 for n≥2, giving Ω(n log n). Exactly n singleton leaves cost Θ(n) in total and do not change the main order. Sizes 0 and 1 cost Θ(1) separately. The argument uses actual intervals, without an unproved monotonicity assumption.

02-E10 Model changes and an early-exit variant

Sorted input causes no insertion shifts, giving Θ(n). Standard MERGE still copies and writes every internal interval, giving Θ(n log n). For the variant, after sorting both halves, A[q] is the left maximum and A[q+1] the right minimum. If their comparison succeeds, concatenation is ordered; equal left records already precede equal right records, preserving stability. On sorted input every merge is skipped. The recursion tree has n leaves and n−1 internal nodes with constant checking work, giving Θ(n). Worst-case time remains O(n log n), since skipping only removes work; interleaved inputs can retain merging costs.

If comparisons require Θ(L) character reads in the worst case, charge each actual comparison accordingly instead of treating every guard as constant. Reference moves may still cost O(1); indiscriminately multiplying every operation by L is not an exact cost model.

Getting started solutions (continued)

Verification code appendix: insertion_sort.py

```
"""Trace insertion sort and verify the inversion count."""

def insertion_trace(values):
    a = list(values)
    moves = 0
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0 and a[j] > key:
            a[j + 1] = a[j]
            j -= 1
            moves += 1
        a[j + 1] = key
    return a, moves


def inversions(values):
    return sum(values[i] > values[j] for i in range(len(values)) for j in range(i + 1,
len(values)))


sample = [7, 2, 6, 1]
sorted_values, moves = insertion_trace(sample)
assert sorted_values == sorted(sample)
assert moves == inversions(sample) == 5
print(sorted_values, moves)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Getting started solutions (continued)

Verification code appendix: sorting_models.py

```
"""Finite independent sorting/count checks; universal arguments are in the chapter."""
from itertools import product,permutations
def insertion(a):
 a=list(a); shifts=comp=guard=writes=0
 for j in range(1,len(a)):
  key=a[j];i=j-1
  while True:
   guard+=1
   if i<0:break
   comp+=1
   if not a[i][0]>key[0]:break
   a[i+1]=a[i];writes+=1;shifts+=1;i-=1
  a[i+1]=key;writes+=1
 return a,(shifts,comp,guard,writes)
def merge_sort(a):
 a=list(a);copies=writes=comps=peak=0
 def go(p,r):
  nonlocal copies,writes,comps,peak
  if r-p<=1:return
  q=p+(r-p+1)//2;go(p,q);go(q,r)
  left=a[p:q];right=a[q:r];copies+=r-p;peak=max(peak,r-p)
  i=j=0;out=[]
  while i<len(left) and j<len(right):
   comps+=1
   if left[i][0]<=right[j][0]:out.append(left[i]);i+=1
   else:out.append(right[j]);j+=1
  out+=left[i:]+right[j:];a[p:r]=out;writes+=r-p
 go(0,len(a));return a,(copies,writes,comps,peak)
if __name__=='__main__':
 count=0
 for n in range(7):
  for keys in product(range(3),repeat=n):
   a=list(zip(keys,range(n)));expected=sorted(a,key=lambda x:x[0])
   inv=sum(keys[i]>keys[j] for i in range(n) for j in range(i+1,n))
   b,c=insertion(a);assert b==expected and c[0]==inv
   assert c[2]==inv+max(0,n-1) and c[3]==c[2]
   b,d=merge_sort(a);assert b==expected and d[0]==d[1]
   count+=1
 assert insertion([(4,'a'),(1,'b'),(3,'c'),(1,'d')])[1]==(4,6,7,7)
 assert merge_sort(list(zip([6,2,5,1,4],'abcde')))[1][0:2]==(12,12)
 assert merge_sort(list(zip([1,2,3,4,5,6],range(6))))[1][0:2]==(16,16)
 assert merge_sort(list(zip(range(8),range(8))))[1][2]==12
 assert max(merge_sort(list(zip(p,range(6))))[1][2] for p in permutations(range(6)))==11
 print(f'PASS: {count} exhaustive repeated-key arrays n<=6; trace counts, stable outputs,
inversion identity; finite only')
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Getting started solutions (continued)

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study